

Finite Mathematical Derivations and Numerical Stability in Multi-Physics Aerospace Simulations

Cartik Sharma

Neuromorph Space Systems, Advanced Computation Division

Abstract

This report provides the rigorous finite mathematical derivations supporting the Space Combustion Physics platform. We detail the discretization of the Navier-Stokes equations for quasi-1D flow, the linearization of pitch-plane dynamics for state-space control, and the implementation of Runge-Kutta numerical integration for high-performance trajectory modeling. These derivations ensure the numerical stability and physical accuracy of the integrated aerospace mission control environment.

1. Finite Difference Discretization of Combustion PDEs

To solve the energy and species transport equations, we apply a central difference scheme. The Laplacian operator for temperature T at node i is approximated as:

$$d^2T/dx^2 \big|_i \approx [T(i+1) - 2T(i) + T(i-1)] / (dx^2)$$

Substituting this into the heat equation with Arrhenius source terms (w):

$$dT/dt = \alpha * [(T_{i+1} - 2T_i + T_{i-1})/dx^2] + Q*w/\rho*C_p$$

2. Linearized State-Space Derivation

The non-linear rocket dynamics $f(x,u)$ are linearized using a first-order Taylor expansion about an equilibrium state (x_{ref}, u_{ref}) :

$$\dot{\delta x} = A * \delta x + B * \delta u$$

$$A_{ij} = df_i / dx_j \big|_{(x_{ref}, u_{ref})}$$

For pitch dynamics, this leads to the state vector $[\theta, q, v, \alpha]^T$ and the matrix A :

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ M_\alpha & M_q & M_v & 0 \\ 0 & 0 & D_v & D_\alpha \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

3. Numerical Stability in CFD (Lax-Friedrichs)

For the hyperbolic conservation laws in nozzle flow, we use the Lax-Friedrichs scheme to maintain stability at the sonic point ($Ma=1$):

$$U_i^{n+1} = 0.5 * (U_{i+1}^n + U_{i-1}^n) - (dt/2dx) * (F_{i+1}^n - F_{i-1}^n) + dt*S_i$$

Stability is maintained if the Courant-Friedrichs-Lewy (CFL) condition is met:

$$CFL = (u + c) * dt / dx \leq 1.0$$

4. High-Performance Integration (RK45)

Trajectory integration uses the 4th-order Runge-Kutta method with error estimation (Dormand-Prince):

$$k_1 = f(t_n, y_n)$$

$$k_2 = f(t_n + 0.5h, y_n + 0.5h*k_1)$$

$$y_{n+1} = y_n + (h/6)*(k_1 + 2k_2 + 2k_3 + k_4)$$

Conclusion

These derivations provide the mathematical infrastructure for the mission control dashboard. By implementing these finite math constructs, we achieve a balance between computational efficiency and high-fidelity physical realism.